

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	ITA0517	Course Title:	Computer Vision
CO Assessed:	CO5 – Naturalization (independent application using OpenCV)P5	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	25% of CIA	Total Marks:	100
CO4 Statement:	Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)		
Student Name:	Yoga Madha A	Reg. No.:	192425058
Section / Batch:	B	Date:	24-08-0226

Code of Conduct

I Yoga Madha A-192425058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	

1. Multi-Image Batch Processing System



1. Problem Understanding

The system is designed to process multiple images stored inside a folder automatically.

Input

- A folder containing multiple image files.
- Supported formats can include JPG, JPEG, PNG, and BMP.

Processing

Each image is processed one at a time using:

1. Image loading
2. Preprocessing
3. Feature detection
4. Feature extraction
5. Result generation
6. Result storage

Output

- Processed images
- Detected features or keypoints
- Image statistics
- Processing status
- A result file containing information about every processed image

Main Constraints

- Batch processing of multiple images
- Efficient memory usage
- Avoid loading all images into memory simultaneously
- Handle invalid or corrupted images
- Store results safely

2. Proposed System

The system follows a modular image-processing pipeline:

Input Folder

- Read Image
- Preprocess Image
- Detect Features
- Extract Features
- Store Result
- Release Memory
- Process Next Image

The important point is that only the current image is kept in memory. After processing and storing its result, the image is released before the next image is loaded.

3. Modules

Module 1 – Folder Scanner

- Reads the input folder.
- Identifies image files.
- Ignores unsupported files.

Module 2 – Image Loader

- Loads one image at a time.
- Checks whether the image was loaded successfully.
- Skips corrupted or unreadable images.

Module 3 – Preprocessing

Typical preprocessing operations are:

- Resize image
- Convert to grayscale
- Noise reduction using Gaussian blur
- Contrast enhancement if required

Preprocessing improves image quality and makes feature detection more reliable.

Module 4 – Feature Detection

A feature detector such as ORB can be used to identify important points in the image.

Examples of detected features:

- Corners
- Edges
- Texture patterns
- Keypoints

Module 5 – Feature Extraction

Feature descriptors are generated for the detected keypoints. These descriptors can later be used for image matching or classification.

Module 6 – Result Storage

The system stores:

- Image name
- Image size
- Number of detected keypoints
- Feature information
- Processing status
- Error information if processing fails

The results can be stored in CSV, JSON, or a database.

4. Main Pseudocode

BEGIN

SET input_folder
SET output_folder
SET result_file

CALL InitializeSystem()

image_files ← GetImageFiles(input_folder)

IF image_files is empty THEN
 DISPLAY "No images found"

```

    STOP
END IF

FOR each image_file IN image_files DO

    DISPLAY "Processing: " + image_file

    image ← LoadImage(image_file)

    IF image is invalid THEN
        StoreError(image_file, "Unable to load image")
        CONTINUE
    END IF

    preprocessed_image ← PreprocessImage(image)

    keypoints, descriptors ← DetectAndExtractFeatures(
        preprocessed_image
    )

    result ← CreateResult(
        image_file,
        keypoints,
        descriptors
    )

    SaveProcessedImage(
        preprocessed_image,
        output_folder
    )

    SaveResult(
        result,
        result_file
    )

    RELEASE image
    RELEASE preprocessed_image
    RELEASE keypoints
    RELEASE descriptors
    RELEASE result

END FOR

CloseResultFile()

DISPLAY "Batch processing completed"

END

```

5. Modular Pseudocode

Module 1 – Get Image Files

FUNCTION GetImageFiles(folder)

 image_files ← empty list

 FOR each file IN folder DO

 IF file extension is JPG

 OR JPEG

 OR PNG

 OR BMP THEN

 ADD file to image_files

 END IF

 END FOR

 RETURN image_files

END FUNCTION

Module 2 – Load Image

FUNCTION LoadImage(file)

 image ← Read image from file

 IF image cannot be read THEN

 RETURN INVALID

 END IF

 RETURN image

END FUNCTION

Module 3 – Preprocess Image

FUNCTION PreprocessImage(image)

 resized_image ← Resize(image)

 gray_image ← ConvertToGrayscale(resized_image)

 filtered_image ← ApplyGaussianBlur(gray_image)

 RETURN filtered_image

END FUNCTION

Module 4 – Feature Detection

FUNCTION DetectAndExtractFeatures(image)

 CREATE ORB feature detector

```
keypoints, descriptors ← ORB_DetectAndCompute(image)
```

```
RETURN keypoints, descriptors
```

```
END FUNCTION
```

Module 5 – Create Result

```
FUNCTION CreateResult(filename, keypoints, descriptors)
```

```
    result.image_name ← filename
```

```
    result.keypoint_count ← NumberOf(keypoints)
```

```
    result.descriptor_count ← NumberOf(descriptors)
```

```
    IF keypoint_count > 0 THEN
```

```
        result.status ← "SUCCESS"
```

```
    ELSE
```

```
        result.status ← "NO FEATURES"
```

```
    END IF
```

```
    RETURN result
```

```
END FUNCTION
```

Module 6 – Store Result

```
FUNCTION SaveResult(result, result_file)
```

```
    WRITE result.image_name
```

```
    WRITE result.keypoint_count
```

```
    WRITE result.descriptor_count
```

```
    WRITE result.status
```

```
END FUNCTION
```

6. Looping Logic

The main FOR loop is responsible for batch processing.

```
FOR each image_file IN image_files
```

```
    Load image
```

```
    IF image is invalid
```

```
        CONTINUE to next image
```

```
    END IF
```

```
    Preprocess image
```

```
    Detect features
```

```
    Store results
```

```
    Release memory
```

END FOR

This allows the same processing operations to be automatically applied to every image in the folder.

7. Memory-Efficient Processing

A simple but important optimization is to process images one at a time.

Inefficient approach

Load Image 1

Load Image 2

Load Image 3

...

Load Image 1000

Process all images

This can consume a large amount of memory.

Efficient approach

Load Image 1

Process Image 1

Save Result 1

Release Image 1

Load Image 2

Process Image 2

Save Result 2

Release Image 2

Load Image 3

Process Image 3

Save Result 3

Release Image 3

Therefore, memory usage remains relatively stable even when the folder contains many images.

8. Error Handling

The system should not stop completely because of one invalid image.

FOR each image

 image ← LoadImage(image)

 IF image is invalid THEN

 Save error information

 CONTINUE

 END IF

 Process image

 Save result

END FOR

For example, if 99 images are valid and the 50th image is corrupted, the system skips the 50th image and continues processing images 51 to 99.

9. Example Result

Image Name	Keypoints	Status
image01.jpg	245	SUCCESS
image02.jpg	318	SUCCESS
image03.jpg	156	SUCCESS
image04.jpg	0	NO FEATURES
image05.jpg	-	ERROR

The result can be stored in a CSV or JSON file for later analysis.

10. Complete Algorithm

BEGIN

```
INPUT_FOLDER ← "input_images"
OUTPUT_FOLDER ← "processed_images"
RESULT_FILE ← "results.csv"
```

CREATE output folder if it does not exist

```
image_files ← GetImageFiles(INPUT_FOLDER)
```

```
IF number of image_files = 0 THEN
    PRINT "No images available"
    STOP
END IF
```

Open RESULT_FILE

```
FOR index ← 0 TO number of image_files - 1 DO
```

```
    current_file ← image_files[index]
```

```
    PRINT "Processing image ", index + 1,
        " of ", number of image_files
```

```
    image ← LoadImage(current_file)
```

```
    IF image = INVALID THEN
```

```
        SaveError(current_file,
            "Image loading failed")
```

```
        CONTINUE
```

```
    END IF
```

```
    processed_image ← PreprocessImage(image)
```

```
    keypoints, descriptors ←
```

```

        DetectAndExtractFeatures(processed_image)

    result ← CreateResult(
        current_file,
        keypoints,
        descriptors
    )

    SaveProcessedImage(
        processed_image,
        OUTPUT_FOLDER
    )

    SaveResult(
        result,
        RESULT_FILE
    )

    DELETE image
    DELETE processed_image
    DELETE keypoints
    DELETE descriptors

END FOR

Close RESULT_FILE

PRINT "All images processed successfully"

END

```

11. Control Structures Used

FOR Loop

Used to process every image in the folder.

IF Condition

Used to check:

- Whether images exist
- Whether an image is valid
- Whether features were detected
- Whether an error occurred

CONTINUE

Used to skip an invalid image and move to the next image.

Functions

Functions divide the system into smaller modules such as:

- GetImageFiles()
- LoadImage()
- PreprocessImage()
- DetectAndExtractFeatures()
- SaveResult()

This improves readability, reuse, and maintenance.

12. Efficiency Considerations

The proposed system is efficient because:

1. Images are processed sequentially.
2. Only the current image is kept in memory.
3. Results are saved immediately.
4. Invalid images are skipped instead of stopping the entire process.
5. Modular functions avoid repeating code.
6. Supported file types are filtered before processing.
7. Temporary image data is released after processing.

A batch-processing pipeline commonly follows the pattern of loading an image, applying processing stages, and saving the result; processing one image at a time is also a useful way to limit memory consumption.

13. Expected Output

For an input folder containing:

input_images/

image1.jpg
image2.jpg
image3.png
image4.jpg
image5.png

The system produces:

processed_images/

image1_processed.jpg
image2_processed.jpg
image3_processed.jpg
image4_processed.jpg
image5_processed.jpg

results.csv

The CSV file contains the processing information for each image.

14. Advantages

- Automatically processes large numbers of images.
- Reduces manual processing.
- Uses modular program design.
- Provides efficient memory usage.
- Handles invalid images.
- Stores processing results systematically.
- Can be extended with different preprocessing and feature-detection techniques.
- Suitable for computer vision datasets and industrial image analysis.

15. Conclusion

The Multi-Image Batch Processing System provides an automated and memory-efficient method for processing images from a folder. The system uses a loop to process each image individually, applies preprocessing and feature detection, stores the results, and releases temporary memory before processing the next image. The modular structure makes the system easy to understand, maintain, and extend. Therefore, the proposed pseudocode satisfies the requirements of batch processing, preprocessing, feature detection, result storage, modularity, looping logic, error handling, and efficient memory usage.